

Lingo

Language Translation Tool

Project Documentation

CodeAlpha Internship - Task 1

Submitted By: Fatima Eman

Student ID: CA/DF1/192788

Full-Stack Language Translation Web App

Table of Contents

Table of Contents	2
1. Project Overview	3
1.1 Original Task Requirements	3
1.2 What Was Built Beyond the Base Requirement	3
2. Features	3
3. Technology Stack.....	4
3.1 Frontend	4
3.2 Backend.....	4
4. Architecture.....	4
5. Per-User History (No Login Required).....	5
6. API Documentation	5
6.1 Example Request — POST /api/translate	5
6.2 Example Response	5
7. Error Handling & Testing	5
8. Setup & Local Installation	6
8.1 Requirements	6
8.2 Steps.....	6
9. Deployment.....	6
9.1 Recommended Host: Render.com.....	6
9.2 Free-Tier Considerations	6
10. Known Limitations	6
11. Possible Future Improvements.....	7

1. Project Overview

Lingo is a full-stack language translation web application built to fulfil CodeAlpha Internship Task 1: a tool where a user can enter text, select source and target languages, and receive a translated result through a translation API. The project extends the base requirement with a production-style architecture: a Flask backend, a persisted per-user history system, auto language detection, text-to-speech, and a fully responsive, custom-designed interface.

The application is structured as a single deployable unit — one Flask server serves both the REST API and the static frontend — so the entire app can be shared as one public URL.

1.1 Original Task Requirements

- Create a user interface where the user can enter text and select source & target languages.
- Use a translation API (e.g. Google Translate API) to process the input.
- Send the text to the API and get the translated response.
- Display the translated text clearly on the screen.
- Optional: add a copy button or text-to-speech feature for better usability.

1.2 What Was Built Beyond the Base Requirement

- A Flask backend (rather than calling a translation API directly from the browser), keeping translation logic server-side.
- Server-persisted, per-user translation history with favorites — no login system required.
- Auto language detection using langdetect.
- Text-to-speech and copy-to-clipboard, as suggested by the optional requirement.
- A fully custom, responsive UI (glassmorphism design) rather than a default/template interface.
- Input validation and error handling on both frontend and backend.
- A single-server architecture designed for one-URL public deployment.

2. Features

Area	Details
Language selection	Searchable dropdown with flags and native names; auto-detect source language
Translation engine	Google Translate (free, via deep-translator library) — no API key required
Layout	Split view on desktop, stacks vertically on mobile
Feedback	Skeleton loader while translating, toast notifications for errors/status
Input tools	Character counter (2000 max), word counter (CJK-aware), paste-and-clean, clear button
Output tools	Copy to clipboard, text-to-speech (browser native, locale-matched voices)

History	Server-persisted per-user history with favorites — no login required
Swap	Swap source/target languages; resolves "Detect language" using the last detected language
Shortcuts	Ctrl+Enter to translate, Ctrl+K to clear

3. Technology Stack

3.1 Frontend

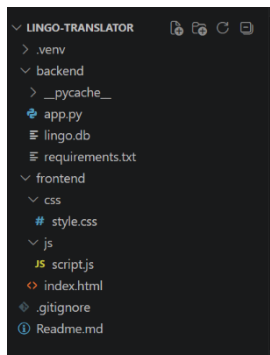
- HTML5, CSS3 (custom glassmorphism design system, no framework)
- Vanilla JavaScript (no build step, no framework dependency)
- Browser-native Web Speech API for text-to-speech

3.2 Backend

- Python 3 + Flask — REST API and static file serving
- deep-translator — free, unofficial Google Translate client
- langdetect — automatic source language detection
- SQLite — lightweight embedded database for per-user history
- Flask-CORS — cross-origin support
- Gunicorn — production WSGI server (used for deployment)

4. Architecture

The project is organized into two top-level folders — a frontend and a backend — with the backend also responsible for serving the frontend's static files in both local development and production.



Request flow for a translation:

1. User types text and selects languages in the browser (frontend/js/script.js).
2. Frontend sends a POST request to /api/translate with the text, source, and target.
3. Flask backend validates the input, optionally auto-detects the source language, and calls Google Translate via deep-translator.
4. The translated text is returned as JSON and rendered in the UI.

- 5. The translation is also saved to that browser's history via a second request to /api/history.

5. Per-User History (No Login Required)

A key design goal was that each visitor's history should be saved for them individually — not shared with other users, and not something the developer can see. This was solved without requiring user accounts:

- On first visit, the browser generates a random unique ID using `crypto.randomUUID()`.
- That ID is stored in the browser's own `localStorage` (key: `lingo_client_id`) and persists across sessions.
- Every history request sends this ID, and the backend scopes all database reads/writes to it.
- A user requesting another user's history or attempting to modify another user's entry receives a "not found" response — verified during testing.

Trade-off: because the ID lives in `localStorage` rather than a real account, history is tied to a specific browser/device. Clearing browser data or switching browsers starts a fresh history. Supporting the same history across multiple devices would require adding a real authentication system.

6. API Documentation

Method	Path	Purpose
POST	/api/translate	Translate text (Google Translate via deep-translator)
GET	/api/history?user_id=...	Fetch a user's saved translation history (last 50)
POST	/api/history	Save a new translation to that user's history
PATCH	/api/history/<id>	Toggle favorite on one history entry
DELETE	/api/history/<id>	Delete one history entry
GET	/api/health	Health check

6.1 Example Request — POST /api/translate

```
{ "text": "Hello, how are you?", "source": "en", "target": "ur" }
```

6.2 Example Response

```
{ "translation": "ہیلو، آپ کیسے ہیں؟", "detectedLanguage": "", "phonetic": "", "alternatives": [] }
```

7. Error Handling & Testing

The following cases were explicitly tested against the backend:

- Empty text input → 400 error, "text is required"
- Unsupported/invalid language code → 400 error, naming the unsupported code
- Malformed JSON request body → 400 error, does not crash the server
- Text exceeding 2000 characters → 400 error, rejected before calling the translation service

- Same source and target language → skips the translation call entirely, returns text unchanged (avoids wasting a request)
- Translation service network failure → 502 error with a friendly message; frontend shows a toast instead of failing silently
- Cross-user history access (wrong user_id on someone else's entry) → rejected with "not found"
- Invalid/malformed user_id on history requests → 400 error

8. Setup & Local Installation

8.1 Requirements

- Python 3.9 or later
- pip

8.2 Steps

```
cd backend pip install -r requirements.txt python app.py
```

Then open <http://127.0.0.1:5000> in a browser. One server serves both the UI and the API — no separate frontend server is needed. The SQLite database file (lingo.db) is created automatically on first run.

9. Deployment

The app is packaged for single-service deployment (frontend + backend together) so it can be hosted as one public URL. A Dockerfile is included for container-based hosts.

9.1 Recommended Host: Render.com

- Push the project to a GitHub repository.
- Create a free account on Render.com and connect the GitHub repository.
- Build Command: `pip install -r backend/requirements.txt`
- Start Command: `cd backend && gunicorn app:app`
- Instance Type: Free

Render's free tier requires a card for identity verification (a refunded \$1 authorization) but does not charge unless usage exceeds free-tier limits.

9.2 Free-Tier Considerations

- Free instances sleep after ~15 minutes of inactivity; the first request after idle takes 30-60 seconds to respond.
- SQLite storage on Render's free tier is not guaranteed to persist across deploys — acceptable for a demo/submission, not for long-term production data.
- The unofficial free Google Translate endpoint (via deep-translator) can rate-limit under heavy shared traffic — acceptable for a class project, not for large-scale production use.

10. Known Limitations

- No alternative translation suggestions or phonetic/romanized output — the free Google Translate tier does not provide these.
- Text-to-speech quality depends on voices installed on the user's OS/browser; if no matching voice exists for a language, the app shows a toast rather than failing silently.
- History is tied to a browser (via localStorage + SQLite), not a real user account.
- Uses an unofficial, free Google Translate endpoint rather than the official paid Google Cloud Translation API — appropriate for a student project, not for commercial production traffic.

11. Possible Future Improvements

- Real user accounts (so history syncs across devices, not just one browser).
- Formal/casual tone rewriting via a secondary LLM pass (attempted during development; removed for this submission to keep the project dependency-free and fully working without any external API key).
- Migrate from SQLite to a managed database (e.g. PostgreSQL) for reliable persistence in production.
- Upgrade to the official, paid Google Cloud Translation API for guaranteed uptime and rate limits at scale.
- Add automated tests (pytest) covering the API endpoints documented in Section 6.

12. User Interface

